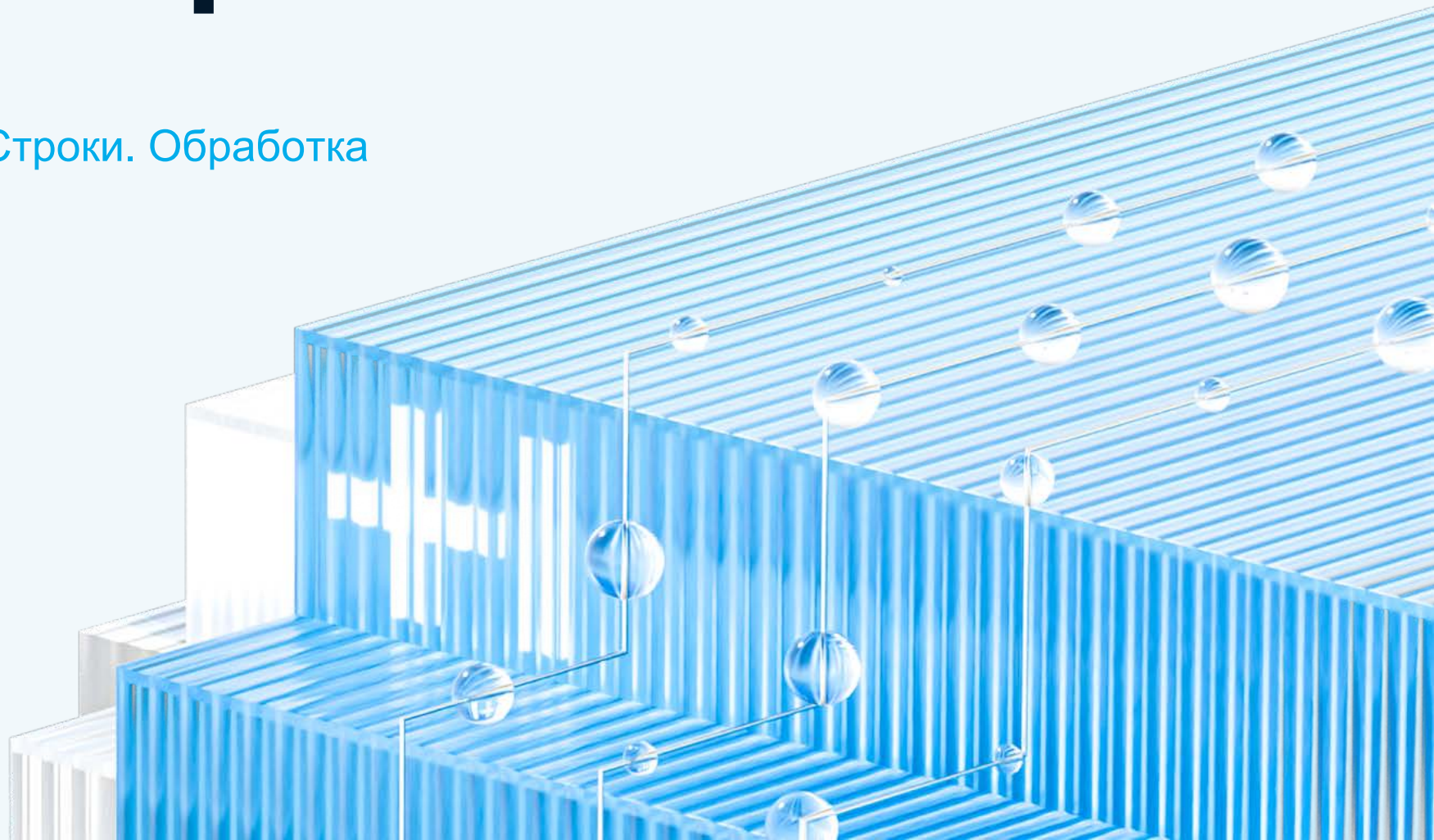
# Основы программирования

23.10.2026

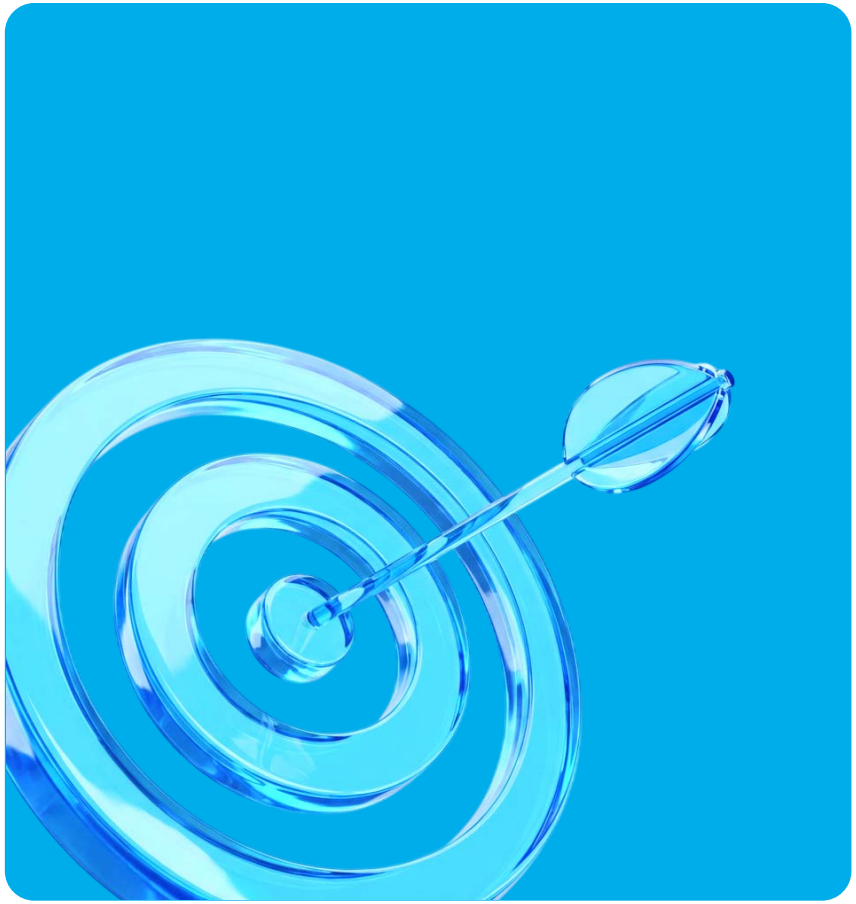
Лабораторная работа № 6. Строки. Обработка текстовой информации.



Проклов Роман Сергеевич

ITI

# Цели лабораторной работы



1. **Познакомиться** со строками как типом данных в Python и C++
2. **Научиться** создавать строки, выполнять базовые операции
3. **Изучить** методы строк: поиск, замена, разбиение, форматирование
4. **Освоить** срезы строк, регулярные выражения (кратко)
5. **Применить** полученные знания для обработки текстовой информации
6. **Отработать** навыки работы с Git: коммиты, ветвление, push

**Формируемые компетенции:** ППК-Р1.1, ППК-Р1.4, ППК-Р1.5

# План лабораторной работы

**Часть 1. Теория: строки**

1. Что такое строка? Создание строк и C-строки
2. Основные операции со строками
3. Методы строк: поиск, замена, разбиение
4. Форматирование строк: f-строки, format()
5. Срезы строк

**Часть 2. Теория: обработка текста**

6. Разбиение текста на слова
7. Подсчёт статистик
8. Поиск и замена
9. Регулярные выражения (краткое введение)

**Часть 4. Практика**

10. Задания на закрепление
11. Работа в мини-группах
12. Сдача отчёта и защита



# Строки в Python

# Строки: последовательность символов

**Строка** — это последовательность символов, заключённая в кавычки.

## Создание строк в Python:

```
# Одинарные кавычки
s1 = 'Привет’

# Двойные кавычки
s2 = "Мир"

# Тройные кавычки (многострочные)
s3 = """Это
многострочная
строка"""

s4 = '''Тоже
многострочная'‘’

# Пустая строка
empty = ""

# Экранирование
escaped = "Он сказал: \"Привет!\""
newline = "Первая строка\nВторая строка"
tab = "Столбец1\tСтолбец2"
backslash = "C:\\Users\\Ivan"
```

## Спецсимволы:

| Символ | Значение |
|---|---|
| \n | Новая строка |
| \t | Табуляция |
| \\ | Обратный слэш |
| \" | Двойная кавычка |
| \' | Одинарная кавычка |

## Строки в Python — неизменяемые:

```
s = "Привет"
# s[0] = "п" # ОШИБКА! Строки неизменяемы
s = "п" + s[1:] # Правильно: создаём новую строку
```

## Длина строки:

```
text = "Привет, мир!"
print(len(text)) # 13
```

# Операции со строками в Python

## Конкатенация (сложение):

```
greeting = "Привет"
name = "Иван"
message = greeting + ", " + name + "!"
print(message)          # Привет, Иван!
```

## Повторение:

```
line = "-" * 20
print(line) # --------------------

pattern = "ab" * 3
print(pattern)          # abababab
```

## Проверка наличия:

```
text = "Привет, мир!"

print("мир" in text)      # True
print("Мир" in text)      # False (регистрозависимо)
print("abc" not in text) # True
```

## Сравнение строк:

```
s1 = "apple"
s2 = "banana"

print(s1 == s2)         # False
print(s1 < s2)          # True (лексикографическое сравнение)
print(s1 > s2)          # False

# Сравнение по кодам символов
print(ord('a'))         # 97
print(ord('A'))         # 65
print(chr(97))          # 'a'
```

## Доступ к символам:

```
text = "Python"
print(text[0])          # P
print(text[1])          # y
print(text[-1])         # n (последний)
print(text[-2])         # o
```

## Итерация по строке:

```
for char in "Python":
    print(char)
```

# Методы строк в Python: поиск и замена

## Поиск:

```
text = "Python — это здорово!"

# find — индекс первого вхождения (-1, если не найдено)
print(text.find("это"))          # 10
print(text.find("Java"))         # -1

# index — как find, но вызывает ошибку, если не найдено
print(text.index("это"))         # 10
# print(text.index("Java"))      # ValueError!

# rfind — индекс последнего вхождения
print(text.rfind("о"))           # 18

# count — количество вхождений
print(text.count("о"))           # 3

# startswith / endswith — начинается/заканчивается
print(text.startswith("Python"))  # True
print(text.endswith("!"))         # True
```

## Замена:

```
text = "Я люблю Python. Python — лучший язык."

# replace — замена всех вхождений
new_text = text.replace("Python", "C++")
print(new_text)          # Я люблю C++. C++ — лучший язык.

# replace с ограничением
new_text2 = text.replace("Python", "C++", 1)
print(new_text2)         # Я люблю C++. Python — лучший язык.
```

## Регистр:

```
text = "Hello, World!"
print(text.upper())              # HELLO, WORLD!
print(text.lower())              # hello, world!
print(text.capitalize())         # Hello, world!
print(text.title())              # Hello, World!
print(text.swapcase())           # hELLO, wORLD!
```

## Удаление пробелов:

```
text = "    Привет, мир!    "

print(text.strip())      # "Привет, мир!"
print(text.lstrip())     # "Привет, мир!    "
print(text.rstrip())     # "    Привет, мир!"
```

# Методы строк в Python: разбиение и объединение

## Разбиение (split):

```
# По пробелам (по умолчанию)
text = "Python это здорово"
words = text.split()
print(words)              # ['Python', 'это', 'здорово']

# По конкретному разделителю
csv = "Иван,25,Москва"
fields = csv.split(",")
print(fields)             # ['Иван', '25', 'Москва']

# С ограничением
text = "a-b-c-d-e"
parts = text.split("-", 2)
print(parts)              # ['a', 'b', 'c-d-e']

# splitlines — разбиение по строкам
multiline = "Строка 1\nСтрока 2\nСтрока 3"
lines = multiline.splitlines()
print(lines)              # ['Строка 1', 'Строка 2', 'Строка 3']
```

## Объединение (join):

```
words = ["Python", "это", "здорово"]
text = " ".join(words)
print(text)               # Python это здорово

csv = ",".join(["Иван", "25", "Москва"])
print(csv)                # Иван,25,Москва

# С пустым разделителем
letters = "".join(["П", "р", "и", "в", "е", "т"])
print(letters)            # Привет
```

## Разбиение и объединение вместе:

```
text = " много пробелов здесь "

# Очистка и нормализация
words = text.split()                  # ['много', 'пробелов', 'здесь']
cleaned = " ".join(words)             # 'много пробелов здесь'
print(cleaned)
```

## Применение:

- Парсинг CSV-файлов
- Обработка логов
- Разбиение предложений на слова
- Нормализация текста

# Форматирование строк в Python

## F-строки (Python 3.6+) — рекомендуемый способ:

```
name = "Иван"
age = 25
pi = 3.14159

print(f"Привет, {name}! Вам {age} лет.")
print(f"Число π: {pi:.2f}")                    # 3.14
print(f"Процент: {0.856:.1%}")                 # 85.6%
print(f"Ширина: {42:5d}")                      # "  42"
print(f"Выравнивание: {42:<5d}")               # "42  "
print(f"Выравнивание: {42:>5d}")               # "  42"

# Выражения в f-строках
a, b = 5, 3
print(f"{a} + {b} = {a + b}")                  # 5 + 3 = 8

# Вызов методов
text = "привет"
print(f"{text.upper()}")                       # ПРИВЕТ

# Вложенные кавычки
print(f"{'текст' in 'Это текст'}")             # True
```

## Метод format() (старый способ):

```
name = "Иван"
age = 25

print("Привет, {}! Вам {} лет.".format(name, age))
print("Привет, {0}! Вам {1} лет.".format(name, age))
print("Привет, {n}! Вам {a} лет.".format(n=name, a=age))
```

## Оператор % (устаревший):

```
print("Привет, %s! Вам %d лет." % (name, age))
```

## Многострочные f-строки:

```
name = "Иван"
age = 25

info = f"""
Имя: {name}
Возраст: {age}
"""
print(info)
```

# Срезы строк в Python

## Синтаксис среза:

```
строка[начало:конец:шаг]
```

## Примеры:

```
text = "Python Programming"

# Базовые срезы
print(text[0:6])        # Python
print(text[:6])         # Python (с начала)
print(text[7:])         # Programming (до конца)
print(text[:])          # Python Programming (копия)
print(text[7:10])       # Pro

# Отрицательные индексы
print(text[-11:])       # Programming (последние 11)
print(text[:-12])       # Python (всё, кроме последних 12)

# С шагом
print(text[::2])        # Pto rgamn (каждый второй)
print(text[1::2])       # yhnPormig (каждый второй, начиная с 1)
print(text[::-1])       # gnimmargorP nohtyP (реверс)
```

## Практические примеры:

```
# Реверс строки
s = "Привет" print(s[::-1])          # тевирП

# Проверка палиндрома
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("А роза упала на лапу Азора"))      # True

# Первая буква каждого слова
text = "python programming language"
initials = "".join(word[0].upper() for word in text.split())
print(initials)             # PPL

# Удаление расширения файла
filename = "document.txt"
name = filename[:filename.rfind(".")]
print(name)             # document
```

## Важно:

- Конец среза не включается
- [::-1] — идиоматический реверс
- Срез создаёт новую строку (строки неизменяемы)

# Строки в C++

# Строки в C++: std::string

**std::string** — это класс из стандартной библиотеки C++, представляющий строку.

## Создание строк:

```cpp
#include <iostream>
#include <string>

int main() {
    // Пустая строка
    std::string empty;

    // Из литерала
    std::string s1 = "Привет";
    std::string s2("Мир");
    std::string s3{"Hello"};

    // Копия другой строки
    std::string s4 = s1;

    // Из нескольких символов
    std::string s5(5, 'a');                    // "aaaaa"

    // С конкатенацией
    std::string s6 = std::string("Hello") + " " + "World";
    std::cout << s1 << "\n";                   // Привет
    std::cout << s5 << "\n";                   // aaaaa
    std::cout << s6 << "\n";                   // Hello World

    return 0;
}
```

## C-строки (устаревший способ):

```cpp
// C-строка — массив символов с нулевым байтом в конце
char cstr[] = "Привет";

// Длина через strlen
#include <cstring>
std::cout << strlen(cstr) << "\n";    // 6

// Небезопасно! Нет проверки границ
char buffer[10];
// strcpy(buffer, "Очень длинная строка");    // ОПАСНО!
```

## Современный C++ использует std::string:

- Безопаснее
- Удобнее
- Автоматическое управление памятью

## Длина строки:

```cpp
std::string s = "Привет";
std::cout << s.length() << "\n";      // 6
std::cout << s.size() << "\n";        // 6 (то же самое)
```

# Операции со строками в C++

## Конкатенация (сложение):

```cpp
#include <iostream>
#include <string>

int main() {
    std::string greeting = "Привет";
    std::string name = "Иван";

    // Через +
    std::string message = greeting + ", " + name + "!";
    std::cout << message << "\n";        // Привет, Иван!

    // Через +=
    std::string s = "Hello"; s += " World";
    std::cout << s << "\n";              // Hello World

    // Через append
    s.append("!!!");
    std::cout << s << "\n";              // Hello World!!!

    return 0;
}
```

**Важно:** std::string::npos — специальное значение, означающее «не найдено».

## Сравнение строк:

```cpp
std::string s1 = "apple";
std::string s2 = "banana";

if (s1 == s2) { /* ... */ }
if (s1 < s2) { /* true, 'a' < 'b' */ }
if (s1 != s2) { /* true */ }

// Через compare
int result = s1.compare(s2);
// < 0, если s1 < s2
// = 0, если равны
// > 0, если s1 > s2
```

## Доступ к символам:

```cpp
std::string text = "Python";

std::cout << text[0] << "\n";        // P
std::cout << text.at(1) << "\n";     // y (с проверкой)
std::cout << text.front() << "\n";   // P
std::cout << text.back() << "\n";    // n

// Изменение
text[0] = 'J';
std::cout << text << "\n";           // Jython
```

## Поиск:

```cpp
std::string text = "Python — это здорово!";

// find — индекс первого вхождения
size_t pos = text.find("это");
if (pos != std::string::npos) {
    std::cout << "Найдено на позиции " << pos << "\n";        // 10
}

// rfind — последнее вхождение
size_t last = text.rfind("о");

// find_first_of — первый из набора символов
size_t vowel = text.find_first_of("аеёиоуыэюя");
```

# Методы строк в C++

## Подстрока (substr):

```
std::string text = "Python Programming";

// substr(начало, длина)
std::string sub1 = text.substr(0, 6);   // "Python"
std::string sub2 = text.substr(7);              // "Programming"
std::string sub3 = text.substr(7, 3);  // "Pro"
```

## Вставка и удаление:

```
std::string s = "Hello World";

// insert
s.insert(5, ",");                // "Hello, World"

// erase
s.erase(5, 1);                   // "Hello World"
s.erase(s.begin(), s.begin() + 5); // " World"

// clear
s.clear();                       // ""
```

## Замена:

```
std::string text = "I love Python. Python is great.";

// Замена всех вхождений (вручную)
size_t pos = 0;
while ((pos = text.find("Python", pos)) != std::string::npos) {
      text.replace(pos, 6, "C++");
      pos += 3;                                  // длина "C++"
}
std::cout << text << "\n";   // I love C++. C++ is great.
```

## Регистр (через std::transform):

```
#include <algorithm>
#include <cctype>

std::string text = "Hello, World!";

// Верхний регистр
std::string upper = text;
std::transform(upper.begin(), upper.end(), upper.begin(),
                                    [](unsigned char c) { return std::toupper(c); });
std::cout << upper << "\n";                    // HELLO, WORLD!

// Нижний регистр
std::string lower = text;
std::transform(lower.begin(), lower.end(), lower.begin(),
                                    [](unsigned char c) { return std::tolower(c); });
```

## Удаление пробелов:

```
std::string text = "  Hello  ";

// Слева
text.erase(0, text.find_first_not_of(" \t\n\r"));

// Справа
text.erase(text.find_last_not_of(" \t\n\r") + 1);

std::cout << "[" << text << "]\n";             // [Hello]
```

## Разбиение строки:

```
#include <sstream> #include <vector>

std::string text = "Python это здорово";
std::vector<std::string> words;
std::istringstream iss(text);
std::string word;

while (iss >> word) {
      words.push_back(word);
}
// words = {"Python", "это", "здорово"}
```

# Форматирование строк в C++

## std::cout с манипуляторами:

```cpp
#include <iostream> #include <iomanip> #include <string>

int main() {
    std::string name = "Иван";
    int age = 25;
    double pi = 3.14159;

    // Простой вывод
    std::cout << "Привет, " << name << "! Вам " << age << " лет.\n";

    // Форматирование чисел
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Число π: " << pi << "\n";              // 3.14

    // Ширина поля
    std::cout << std::setw(5) << 42 << "\n";            // " 42"

    // Выравнивание
    std::cout << std::left << std::setw(5) << 42 << "\n"; // "42 "
    std::cout << std::right << std::setw(5) << 42 << "\n"; // " 42"

    return 0;
}
```

## std::ostringstream — старый способ:

```cpp
#include <sstream> #include <iomanip>

std::ostringstream oss; oss << "Привет, " << name << "! Вам " << age << " лет.";
std::string message = oss.str();
```

## std::format (C++20) — современный способ:

```cpp
#include <format> #include <iostream>

int main() {
    std::string name = "Иван";
    int age = 25;
    double pi = 3.14159;

    // Аналог f-строк Python
    std::string message = std::format("Привет, {}! Вам {} лет.", name, age);
    std::cout << message << "\n";

    // Форматирование чисел
    std::cout << std::format("Число π: {:.2f}\n", pi);    // 3.14
    std::cout << std::format("Ширина: {:5d}\n", 42);    // " 42"

    return 0;
}
```

## Сравнение с Python:

| Python | C++ |
|---|---|
| f"Привет, {name}!" | std::format("Привет, {}!", name) (C++20) |
| f"{pi:.2f}" | std::format("{:.2f}", pi) |
| f"{x:5d}" | std::format("{:5d}", x) |

**Важно:** std::format требует C++20. Если компилятор не поддерживает — используйте std::cout с манипуляторами.

# Python vs C++: сравнение строк

| Операция | Python | C++ |
|---|---|---|
| **Создание** | s = "text" | std::string s = "text"; |
| **Длина** | len(s) | s.length() |
| **Доступ** | s[0], s[-1] | s[0], s.back() |
| **Конкатенация** | s1 + s2 | s1 + s2 |
| **Повторение** | s * 3 | циклическое += |
| **Поиск** | s.find("x") | s.find("x") |
| **Замена** | s.replace(a, b) | s.replace(pos, len, b) |
| **Разбиение** | s.split(",") | std::istringstream |
| **Объединение** | ",".join(lst) | Ручной цикл |
| **Регистр** | s.upper() | std::transform |
| **Срез** | s[1:5] | s.substr(1, 4) |
| **Реверс** | s[::-1] | std::reverse |

**Ключевое различие:**
Python имеет более богатый API для строк.
В C++ многие операции требуют ручной реализации.

## Пример: обработка CSV на Python

```python
line = "Иван,25,Москва"
fields = line.split(",") name, age, city = fields
print(f"{name} ({age}) из {city}")
```

## Пример: обработка CSV на C++

```cpp
#include <iostream>
#include <sstream>
#include <vector>
#include <string>

int main() {
    std::string line = "Иван,25,Москва";
    std::vector<std::string> fields;
    std::istringstream iss(line);
    std::string field;

    while (std::getline(iss, field, ',')) {
        fields.push_back(field);
    }

    std::cout << fields[0] << " (" << fields[1] << ") из "
              << fields[2] << "\n";
    return 0;
}
```

# Обработка текста

# Подсчёт статистик текста

**Задача:** Подсчитать количество слов, символов, предложений, частоту слов.

**Python:**

```python
def text_stats(text: str) -> dict:
    """Возвращает статистику текста."""
    words = text.split() sentences = text.count('.') + text.count('!') + text.count('?')
    chars = len(text)
    chars_no_spaces = len(text.replace(" ", ""))

    # Частота слов
    from collections import Counter
    word_freq = Counter(word.lower().strip(".,!?") for word in words)

    return {
        "words": len(words),
        "sentences": sentences,
        "chars": chars,
        "chars_no_spaces": chars_no_spaces,
        "top_words": word_freq.most_common(3)
    }

text = "Python это здорово. Python прост. Python мощен!"
stats = text_stats(text)

print(f"Слов: {stats['words']}")
print(f"Предложений: {stats['sentences']}")
print(f"Символов: {stats['chars']}")
print(f"Топ-3 слова: {stats['top_words']}")
```

**C++:**

```cpp
#include <iostream> #include <string> #include <sstream>
#include <map> #include <algorithm> #include <cctype>

int main() {
    std::string text = "Python это здорово. Python прост. Python мощен!";
    // Подсчёт слов
    std::istringstream iss(text);
    std::string word;
    std::map<std::string, int> wordFreq;
    int wordCount = 0;

    while (iss >> word) {
        ++wordCount;
        // Убираем пунктуацию
        word.erase(std::remove_if(word.begin(), word.end(),
                                  [](unsigned char c) { return std::ispunct(c); }), word.end());
        // В нижний регистр
        std::transform(word.begin(), word.end(), word.begin(),
                       [](unsigned char c) { return std::tolower(c); });
        wordFreq[word]++;
    }
    // Подсчёт предложений
    int sentences = std::count_if(text.begin(), text.end(),
        [](char c) { return c == '.' || c == '!' || c == '?'; });

    std::cout << "Слов: " << wordCount << "\n";
    std::cout << "Предложений: " << sentences << "\n";
    std::cout << "Символов: " << text.length() << "\n";

    // Топ-3 слова
    std::cout << "Частота слов:\n";
    for (const auto& [w, count] : wordFreq) {
        std::cout << w << ": " << count << "\n";
    }
    return 0;
}
```

# Поиск и извлечение данных из текста

**Задача:** Извлечь email-адреса, URL, телефоны из текста.

## Python - регулярные выражения:

```python
import re

text = """
Контакты:
Email: ivan@example.com, maria@company.ru
Сайт: https://example.com
Телефон: +7 (999) 123-45-67
"""

# Email
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', text)
print(f"Email: {emails}")
# ['ivan@example.com', 'maria@company.ru']

# URL
urls = re.findall(r'https?://[\w./-]+', text)
print(f"URL: {urls}")
# ['https://example.com']

# Телефон
phones = re.findall(r'\+?\d[\d\s()-]{9,}', text)
print(f"Телефоны: {phones}")
# ['+7 (999) 123-45-67']
```

## Python - без регулярных выражений:

```python
def extract_emails(text: str) -> list:
    """Извлекает email-адреса без regex."""
    emails = []
    for word in text.split():
        word = word.strip(".,;:!?()<>[]")
        if "@" in word and "." in word.split("@")[1]:
            emails.append(word)
    return emails
```

## Основные метасимволы regex:

| Символ | Значение |
|---|---|
| \d | Цифра |
| \w | Буква, цифра, _ |
| \s | Пробельный символ |
| . | Любой символ |
| + | Один или более |
| * | Ноль или более |
| ? | Ноль или один |
| [] | Набор символов |
| () | Группа |

## C++ - регулярные выражения (C++11):

```cpp
#include <iostream> #include <string> #include <regex>

int main() {
    std::string text = R"(
Контакты:
Email: ivan@example.com, maria@company.ru
Сайт: https://example.com
)";

    // Email
    std::regex emailPattern(R"([\w.+-]+@[\w-]+\.[\w.]+)");
    std::sregex_iterator it(text.begin(), text.end(), emailPattern);
    std::sregex_iterator end;

    std::cout << "Email:\n";
    for (; it != end; ++it) {
        std::cout << " " << it->str() << "\n";
    }

    // URL
    std::regex urlPattern(R"(https?://[\w./-]+)");
    std::sregex_iterator urlIt(text.begin(), text.end(), urlPattern);

    std::cout << "URL:\n";
    for (; urlIt != end; ++urlIt) {
        std::cout << " " << urlIt->str() << "\n";
    }

    return 0;
}
```

# Практика

# Задание 1: Подсчёт слов в строке

**Задача:** Написать программу, которая подсчитывает количество слов в строке.

```python
def count_words(text: str) -> int:
    """Подсчитывает количество слов в тексте."""
    return len(text.split())

# Использование
text = input("Введите текст: ")
print(f"Количество слов: {count_words(text)}")

# С учётом пунктуации
import re
def count_words_clean(text: str) -> int:
    """Подсчитывает слова, игнорируя пунктуацию."""
    words = re.findall(r'\b\w+\b', text)
    return len(words)

text = "Привет, мир! Как дела?"
print(f"Слов (с пунктуацией): {count_words(text)}")       # 4
print(f"Слов (без пунктуации): {count_words_clean(text)}")  # 4
```

```cpp
#include <iostream>
#include <string>
#include <sstream>

int countWords(const std::string& text) {
    std::istringstream iss(text);
    std::string word;
    int count = 0;
    while (iss >> word) {
        ++count;
    }
    return count;
}

int main() {
    std::string text;
    std::cout << "Введите текст: ";
    std::getline(std::cin, text);

    std::cout << "Количество слов: " << countWords(text) << "\n";
    return 0;
}
```

# Задание 2: Проверка палиндрома

**Задача:** Проверить, является ли строка палиндромом (игнорируя регистр, пробелы, пунктуацию).

```python
import re

def is_palindrome(text: str) -> bool:
    """Проверяет, является ли строка палиндромом."""
    # Очистка: только буквы и цифры, нижний регистр
    cleaned = re.sub(r'[^\w]', '', text.lower())
    return cleaned == cleaned[::-1]

# Тесты
tests = [
    "А роза упала на лапу Азора",
    "Madam, I'm Adam",
    "Hello, World!",
    "12321",
    "racecar"
]

for text in tests:
    result = "палиндром" if is_palindrome(text) else "не палиндром"
    print(f"'{text}' — {result}")
```

**Разбор:**

- Очистка строки: удаляем всё, кроме букв и цифр
- Приводим к нижнему регистру
- Сравниваем с реверсом

```cpp
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>

bool isPalindrome(const std::string& text) {
    // Очистка: только буквы и цифры, нижний регистр
    std::string cleaned;
    for (char c : text) {
        if (std::isalnum(static_cast<unsigned char>(c))) {
            cleaned += std::tolower(static_cast<unsigned char>(c));
        }
    }

    // Сравнение с реверсом
    std::string reversed = cleaned;
    std::reverse(reversed.begin(), reversed.end());
    return cleaned == reversed;
}

int main() {
    std::string tests[] = {
        "А роза упала на лапу Азора",
        "Madam, I'm Adam",
        "Hello, World!",
        "12321",
        "racecar"
    };

    for (const auto& text : tests) {
        std::cout << "'" << text << "' — "
                  << (isPalindrome(text) ? "палиндром" : "не палиндром")
                  << "\n";
    }
    return 0;
}
```

# Задание 3: Шифр Цезаря

**Задача:** Реализовать шифр Цезаря — сдвиг каждой буквы на K позиций.

```python
def caesar_cipher(text: str, shift: int) -> str:
    """Шифрует текст шифром Цезаря."""
    result = []
    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            shifted = (ord(char) - base + shift) % 26 + base
            result.append(chr(shifted))
        else:
            result.append(char)
    return "".join(result)

# Шифрование
original = "Hello, World!"
encrypted = caesar_cipher(original, 3)
print(f"Оригинал: {original}")
print(f"Зашифровано: {encrypted}")  # Khoor, Zruog!

# Дешифрование (сдвиг в обратную сторону)
decrypted = caesar_cipher(encrypted, -3)
print(f"Расшифровано: {decrypted}")  # Hello, World!
```

```cpp
#include <iostream>
#include <string>

std::string caesarCipher(const std::string& text, int shift) {
    std::string result = text;
    for (char& c : result) {
        if (std::isalpha(static_cast<unsigned char>(c))) {
            char base = std::isupper(static_cast<unsigned char>(c)) ? 'A' : 'a';
            c = (c - base + shift % 26 + 26) % 26 + base;
        }
    }
    return result;
}

int main() {
    std::string original = "Hello, World!";
    std::string encrypted = caesarCipher(original, 3);
    std::string decrypted = caesarCipher(encrypted, -3);

    std::cout << "Оригинал: " << original << "\n";
    std::cout << "Зашифровано: " << encrypted << "\n";  // Khoor, Zruog!
    std::cout << "Расшифровано: " << decrypted << "\n"; // Hello, World!

    return 0;
}
```

**Разбор:**

- Для каждой буквы: (код - base + shift) % 26 + base
- base — код 'A' (65) для заглавных, 'a' (97) для строчных
- Не-буквы остаются без изменений

# Задание 4: Валидация данных

**Задача:** Проверить корректность email, телефона, URL.

```python
import re

def validate_email(email: str) -> bool:
    """Проверяет корректность email."""
    pattern = r'^[\w.+-]+@[\w-]+\.[\w.-]+$'
    return bool(re.match(pattern, email))

def validate_phone(phone: str) -> bool:
    """Проверяет корректность российского телефона."""
    pattern = r'^\+?7[\s-]?\(?\d{3}\)?[\s-]?\d{3}[\s-]?\d{2}[\s-]?\d{2}$'
    return bool(re.match(pattern, phone))

def validate_url(url: str) -> bool:
    """Проверяет корректность URL."""
    pattern = r'^https?://[\w.-]+\.[a-z]{2,}(/[\w./-]*)?$'
    return bool(re.match(pattern, url))

# Тесты
tests = [
    ("email", "ivan@example.com", validate_email),
    ("email", "invalid@", validate_email),
    ("phone", "+7 (999) 123-45-67", validate_phone),
    ("url", "https://example.com", validate_url),
    ("url", "not-a-url", validate_url),
]

for type_, value, validator in tests:
    result = "✓" if validator(value) else "✗"
    print(f"{result} {type_}: {value}")
```

```cpp
#include <iostream>
#include <string>
#include <regex>

bool validateEmail(const std::string& email) {
    std::regex pattern(R"(^[\w.+-]+@[\w-]+\.[\w.-]+$)");
    return std::regex_match(email, pattern);
}

bool validatePhone(const std::string& phone) {
    std::regex pattern(R"(^\+?7[\s-]?\(?\d{3}\)?[\s-]?\d{3}[\s-]?\d{2}[\s-]?\d{2}$)");
    return std::regex_match(phone, pattern);
}

bool validateUrl(const std::string& url) {
    std::regex pattern(R"(^https?://[\w.-]+\.[a-z]{2,}(/[\w./-]*)?$)");
    return std::regex_match(url, pattern);
}

int main() {
    std::cout << std::boolalpha;

    std::cout << "Email valid: " << validateEmail("ivan@example.com") << "\n";   // true
    std::cout << "Email invalid: " << validateEmail("invalid@") << "\n";         // false
    std::cout << "Phone valid: " << validatePhone("+7 (999) 123-45-67") << "\n"; // true
    std::cout << "URL valid: " << validateUrl("https://example.com") << "\n";    // true

    return 0;
}
```

# Задание 5: Форматирование таблицы

**Задача:** Вывести данные в виде красиво отформатированной таблицы.

```python
students = [
    ("Иван", 25, 4.5),
    ("Мария", 22, 4.85),
    ("Пётр", 30, 4.2),
    ("Анна", 20, 4.95),
]

# Заголовок
print(f"{'Имя':<10} {'Возраст':>8} {'Балл':>8}")
print("-" * 28)

# Данные
for name, age, gpa in students:
    print(f"{name:<10} {age:>8} {gpa:>8.2f}")

# Итого
print("-" * 28)
avg_gpa = sum(s[2] for s in students) / len(students)
print(f"{'Средний балл':<10} {'':>8} {avg_gpa:>8.2f}")
```

**Ожидаемый вывод:**

| Имя | Возраст | Балл |
|---|---|---|
| Иван | 25 | 4.50 |
| Мария | 22 | 4.85 |
| Пётр | 30 | 4.20 |
| Анна | 20 | 4.95 |
| Средний балл | | 4.62 |

```cpp
#include <iostream>
#include <vector>
#include <tuple>
#include <iomanip>
#include <string>
#include <numeric>

int main() {
    std::vector<std::tuple<std::string, int, double>> students = {
        {"Иван", 25, 4.5},
        {"Мария", 22, 4.85},
        {"Пётр", 30, 4.2},
        {"Анна", 20, 4.95}
    };

    // Заголовок
    std::cout << std::left << std::setw(10) << "Имя"
              << std::right << std::setw(8) << "Возраст"
              << std::setw(8) << "Балл" << "\n";
    std::cout << std::string(28, '-') << "\n";

    // Данные
    std::cout << std::fixed << std::setprecision(2);
    double totalGpa = 0;

    for (const auto& [name, age, gpa] : students) {
        std::cout << std::left << std::setw(10) << name
                  << std::right << std::setw(8) << age
                  << std::setw(8) << gpa << "\n";
        totalGpa += gpa;
    }

    // Итого
    std::cout << std::string(28, '-') << "\n";
    double avgGpa = totalGpa / students.size();
    std::cout << std::left << std::setw(10) << "Средний балл"
              << std::right << std::setw(8) << ""
              << std::setw(8) << avgGpa << "\n";

    return 0;
}
```

# Задание 6. Работа в мини-группах

**Формат работы:** группы по 3-4 человека.
**Задача:** Каждая группа получает свой вариант. Необходимо реализовать программу на двух языках (Python и C++) и сравнить результаты.

**Варианты задач для групп:**

| Вариант | Задача |
|---|---|
| 1 | Подсчитать количество гласных и согласных в тексте |
| 2 | Найти самое длинное слово в тексте |
| 3 | Заменить все вхождения подстроки без учёта регистра |
| 4 | Проверить, является ли строка корректным IP-адресом |
| 5 | Извлечь все числа из текста и найти их сумму |
| 6 | Разбить текст на предложения и вывести каждое с новой строки |
| 7 | Подсчитать частоту каждой буквы в тексте |
| 8 | Удалить все HTML-теги из текста |

**Критерии оценки:**
- Корректность решения на двух языках
- Понимание различий между языками
- Обработка краевых случаев
- Качество форматирования вывода

**Презентация результатов:** 2–3 минуты на группу.

# Требования к отчёту и критерии оценки

## Структура отчёта:

1. Титульный лист
2. Цель работы
3. Выполнение заданий:
   - Листинг кода на двух языках для каждого задания
   - Скриншоты результатов
   - Примеры использования методов строк
   - Результаты групповой работы
4. Ответы на контрольные вопросы
5. Вывод

## Критерии оценки:

| Критерий | Мин. балл | Макс. балл |
|---|---|---|
| Самостоятельная проработка теории, ответы на вопросы | 1 | 2 |
| Корректность выполнения заданий, рабочий код | 3 | 6 |
| Качество кода (PEP8, структура, комментарии) | 1 | 2 |
| **ИТОГО** | **5** | **10** |

**Формат сдачи:** отчёт в формате PDF или DOCX + код в Git-репозитории (не менее 3 коммитов).

# Контрольные вопросы

# Контрольные вопросы

**Вопросы для самопроверки:**

1. **Что такое строка?** Как создать строку в Python и C++?
2. **Почему строки в Python неизменяемы?** Что это значит на практике?
3. **Как разбить строку на слова в Python?** А в C++?
4. **Что такое срез строки?** Как работает s[::-1]?
5. **Как отформатировать число с двумя знаками после запятой?** В Python и C++?
6. **Как найти подстроку в строке?** Что возвращает find?
7. **Что такое регулярные выражения?** Приведите пример использования.
8. **Как в C++ преобразовать строку в число?** А число в строку?



**Ответы должны быть в отчёте.**

# Задания для самостоятельной работы

# Задания для самостоятельной работы

**Дополнительные задания (для продвинутых):**

1. **Парсер логов:** Извлечь из лог-файла все записи уровня ERROR с временными метками.

2. **Анализ текста:** Подсчитать частоту слов в тексте, вывести топ-10.

3. **Шифрование:** Реализовать шифр Виженера (усложнённый шифр Цезаря).

4. **Валидация:** Проверить корректность JSON-строки (упрощённо).

5. **Сравнение языков (*):** Напишите одинаковую программу на Python и C++ и сравните:
   - Количество строк кода
   - Читаемость
   - Удобство работы со строками



**Дополнительно:**
- Создать README.md в репозитории

# Спасибо
# за внимание

